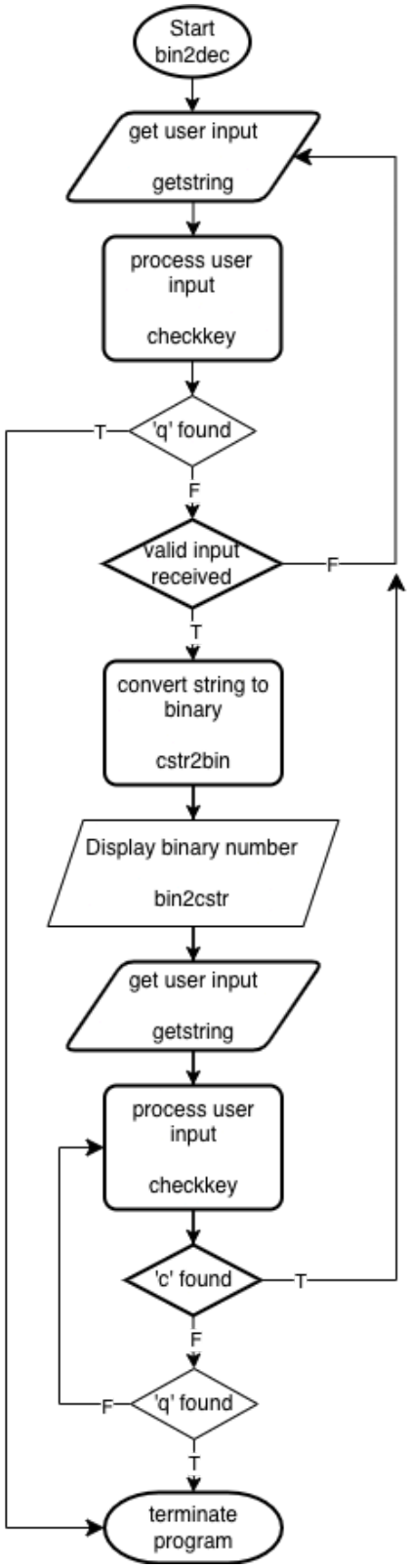
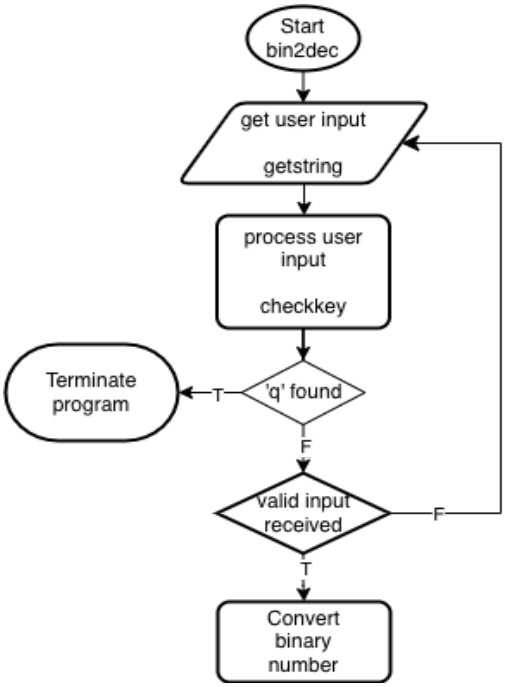
Team Members: Mitch Merell and Osvaldo Medina

CS3B Group Project: Bin2Dec

Overview:

Bin2Dec is a program where the user is, when prompted, able to input a binary number to be converted into its decimal equivalent. The string can consist of 1's, 0's, 'c's, and 'q's; upon pressing 'Enter' the string will be processed: moving from left to right through the string any sequence of uninterrupted binary digits are saved; upon finding a 'c' in the string the previously saved digits will be cleared, and upon finding a 'q' the program will immediately terminate. Upon finding the input where the user entered 'Enter,' the program will then move to convert any saved digits. After the conversion is complete, the program will display the decimal equivalent of the last -saved uninterrupted string of binary digits, with the relevant sign of the number displayed. Once this number is displayed, the program will prompt the user to either clear the number or quit ('c' or 'q'). If a 'c' is input, the program will go back to the initial prompt and the process can begin again; if this input is a 'q', the program will immediately terminate.

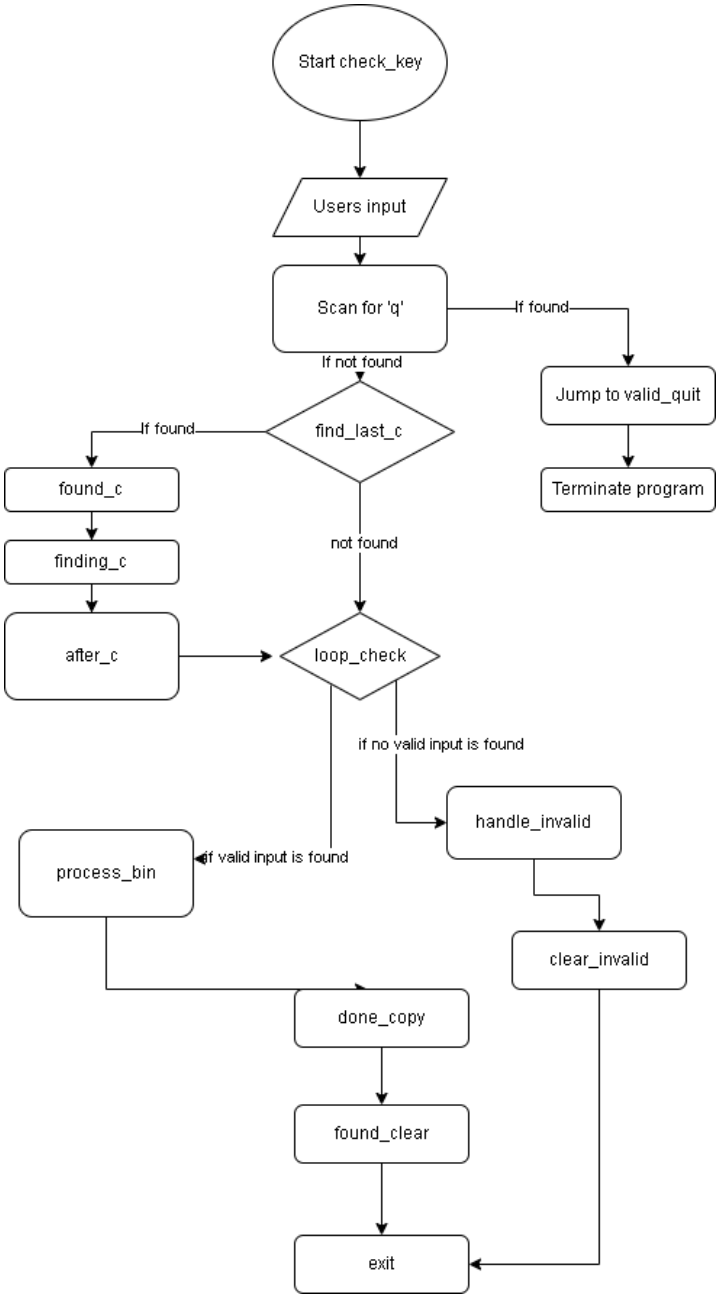




Start:

Upon starting the program, bin2dec will prompt the user for input by displaying a premade C-String through our putstring function. The maximum length of the buffer to receive the input is 64 bytes, including the null terminator; this 64 limit is passed along with the input buffer to getstring(), which receives the input by calling read(). The result from getstring() is a null-terminated string of at most 64 characters, including a null terminator. This C-String is then passed to check_key() to be processed; the result of check_key() will determine the rest of the flow for bin2dec.

check_key():



This is the check_key that is called after the user inputs a string of binary digits in the bin2dec program that is saved in the buffer. Initializing the size of the buffer, read, and write buffer. Afterwards, search_q reads the index to search for ‘q’. Within search_q, this is a loop that loads the byte and checks for ‘q’, If found then will jump to valid_quit to terminate the program immediately. If not found within the index then it will jump to no_q. Within no_q will reset the read index in order to now search for a ‘c’ which will clear anything within the buffer before or at the found ‘c’. Now in the find_last_c function, it's a loop that searches for a ‘c’ and if found, it jumps to found_c to set a flag at the position of ‘c’. By setting a flag in this function it records its position within the buffer by storing it into the index. After which it jumps back to find_last_c to look for another ‘c’ once encountered with the null terminator, now jumps to the finding_c function, where it looks to see if it found ‘c’ characters. If found, then it will jump to after_c to copy anything after the buffer is cleared while clearing anything before the ‘c’ was found. Then jump to loop_check. If no ‘c’ is found, then it will copy the entire buffer and go into loop_check. Within loop_check, this loads in the current char and searches for valid input/binary numbers such as ‘1’s or ‘0’s. If they are found, then it jumps to process_bin, and if no valid input is found then jumps to handle_invalid. Within process_bin the

valid characters are written into the index, and once all valid characters are written within the index, it will jump back to loop_check to find for '\n'/newline to jump to done_copy. Within handle_invalid, it will clear the buffer for invalid inputs and write a null byte to the buffer for these invalid inputs. Afterwards exit this function by jumping to exit. Now in done_copy, if no binary digits are found, then it will jump to handle_invalid and will look to see if any 'c's are found to jump into found_clear. Within found_clear, has the clear code #0 is found, exit the function and returns back into the bin2dec program. Lastly, if no 'c' / clear was for then jump to exit, this restores X29 and X30 from stack then returns from the check_key function into the bin2dec program.

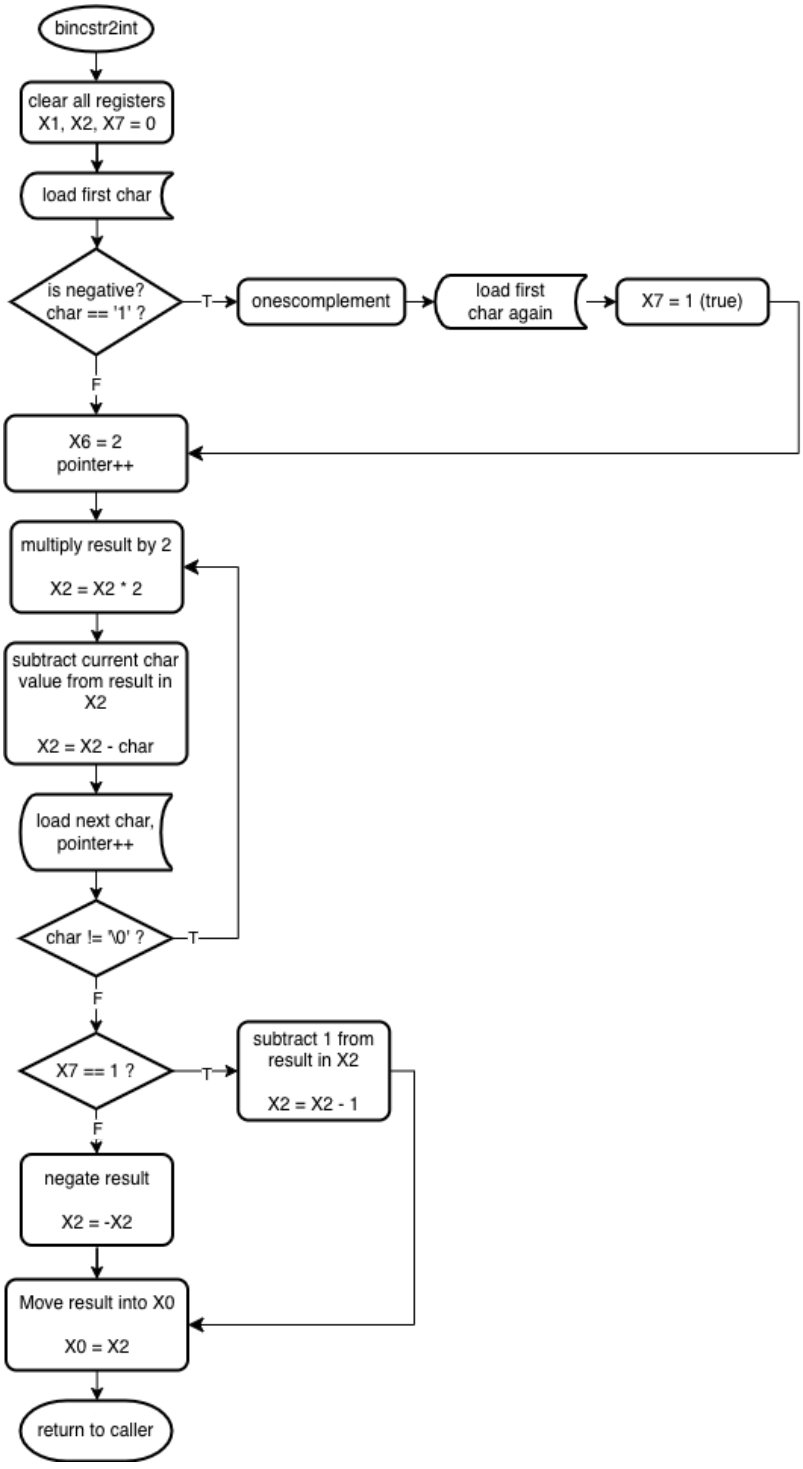
… Back to bin2dec:

Upon returning to the main driver of bin2dec, the return code from check_key() is analyzed. If check_key() returns a value of 1, then the user wants to quit the program, and so bin2dec jumps to the end to prepare to terminate the program. If check_key() returns a value of 0, then a valid consecutive sequence of binary digit characters was found; bin2dec then moves on to passing the parsed C-String to bincstr2int() to be converted into an integer value. If check_key() returns any other value then the input was invalid; the program simply jumps back to the beginning and displays the initial prompt again.

bincstr2int():

This function is responsible for converting the C-String of binary digits produced from check_key() into its signed integer equivalent. In order to process the binary number correctly, incstr2int assumes that all characters within the string passed to this function are strictly binary digits (‘1’ or ‘0’), as well as a null terminator.

Bincstr2int begins by clearing the registers the function will use in order to get rid of unnecessary data leftover from other functions, and loads the first character to set the sign of the resulting decimal number. Since the binary number is a 2’s complement representation, this first character (equivalent to the most significant bit of a binary number) indicates the sign of the number; a ‘1’ is a

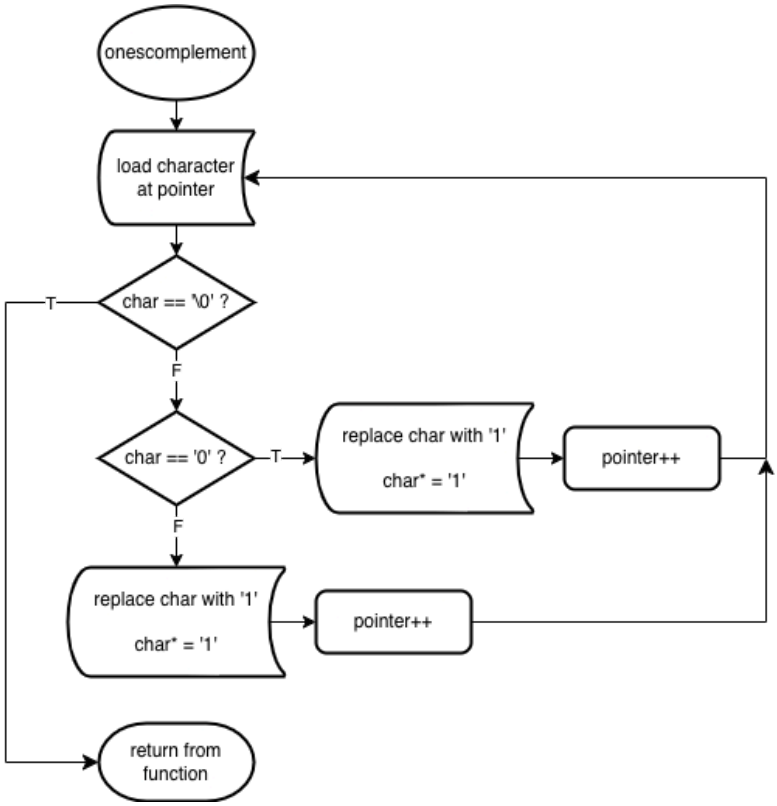


negative and a ‘0’ is a positive. If the number is positive, nothing needs to be done for now; if the number is negative the program needs a way to begin processing it as if it was positive, and so the negative number is then passed to onescomplement() to convert it into its 1’s complement equivalent (all bits are flipped: 1’s to 0’s and 0’s to 1’s). Once the buffer has been converted into its 1’s complement equivalent, bincstr2int loads the first character again into X1 to point at the start of the buffer (which has now had its sign flipped) and sets the “negative” flag in X7 to be 1 (true).

Returning to the main flow of bincstr2int, the function then moves into a loop which performs the actual conversion which runs while the currently loaded character does not have a value of zero, which signifies the null terminator, and negatively accumulates the decimal conversion in X2. In each iteration of the loop, the following sequence is performed for the currently loaded character of the iteration. The negatively accumulated number in X2 is multiplied by 2 (binary numbers are in base 2) which shifts the digits of the number left by one. To get the numerical value of the currently loaded character, the ASCII value of the character ‘0’ is subtracted from the ASCII value of the current character. Finally, this numerical value is then subtracted from the accumulated number in X2. Lastly the next character is then loaded into X1 and compared to a value of 0 to check for the null terminator. If the null terminator is found, the function then exits the loop; if not, the function will move back to the beginning of this loop.

Upon exiting the loop, within X2 there will either be a negated quantity of a positive number we want or a negated one’s complement equivalent depending on the sign of the original number. X7 holds the sign flag, and so if the value in X7 is a 1, the original number was negative; to get back to the original number from a one’s complement that has been negated, we just need to subtract a 1 (or add a -1) to the value in X2. If X7 is 0 then the original number was positive and so we simply need to negate the value in X2. Once the function performs the correct operation on the decimal number in X2, the value is then moved into X0 to be used and the function returns to the caller.

onescomplement():

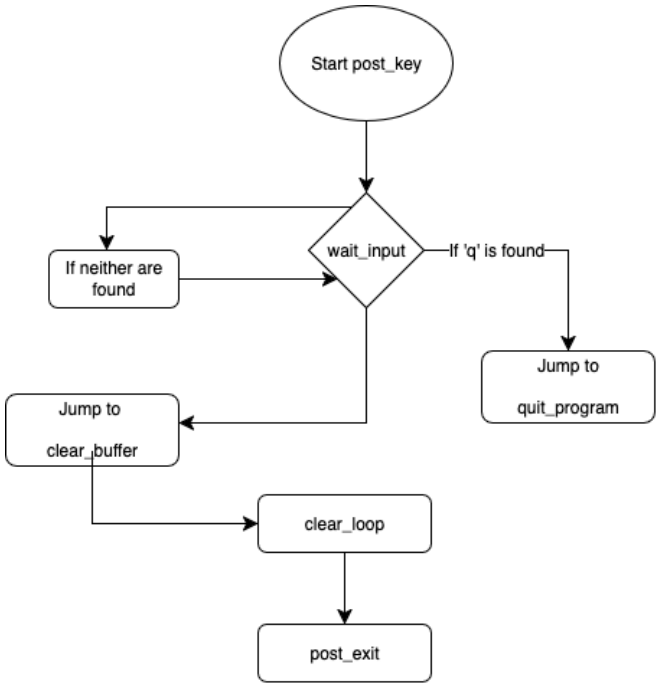


The purpose of this function is, given a pointer to a C-String, to convert the C-String into its 1's complement equivalent: that is, the function will "flip" all of the bits to get a result where all 0's have become 1's and all 1's have become 0's. onescomplement() assumes that the given C-String is a null terminated string consisting only of 1's and 0's.

The bulk of the function is in the beginning loop, where initially the byte currently pointed to by the given pointer is loaded into W1. This character is then compared to a numerical value 0 to check if the loop has reached the end of the string or not. If the character is not null, the loop proceeds to swap ASCII '1's for ASCII '0's and vice-versa. Once the loop encounters a null terminator, the loop is exited and the function returns to the caller.

… Back to bin2dec:

Once the string has been converted to a decimal value, the next step is to display the number itself. The first part of this is displaying an arrow "->" to indicate the separation between the number they input and its decimal equivalent, as well as the sign of the number. Bin2dec checks the sign of the decimal number returned from bincstr2int() by comparing the value to numerical zero; if it's less than zero it's negative and if not then the number is positive. The program then displays a '+' for a positive number; for negative numbers, the '-' is already handled by the int2cstr function. The decimal number is then passed to int2cstr to be displayed to the user.



post_key(): In this function post_key the main responsibility of this function is dealing with the post condition of the user. The user can press ‘c’ for clear or ‘q’ to quit. With quit, it will immediately quit the program once entered, and with clear, it will clear the buffer and jump back into the beginning of the bin2dec program. The wait_input awaits the input that the user is deciding on entering. If the user enters anything other than a ‘c’ or ‘q’ it will not take the input and will wait for a ‘c’ or ‘q’ to be inputted. While with the ‘q’ it will instantly quit the program. Now with ‘c’ being inputted, this will jump to clear_buffer, where it will reset the index to zero. Then jumping to clear_loop, where this will set the byte to 0 on the buffer length so that the buffer is now 0. Afterwards it will jump to post_exit that sets the return value to 0 then exit the program and return to bin2dec.

Program termination:

Upon returning to bin2dec, the value returned from post_key() is then analyzed. If the function returns a value of 0, the user wants to repeat the conversion process and so bin2dec jumps back to the beginning of the program and the initial prompt is shown again. If any other value is returned, the user wishes to quit the program and so bin2dec prepares and executes a supervisor exit() call to terminate the program.